Revisión 1.2

Universidad Nacional de Río Negro

Cloud Computing

Apunte “Kubernetes”

Unidad 4

Docentes:

- Lic. Horacio Munoz Abbate
  - (Horacio Muñoz Abbate)

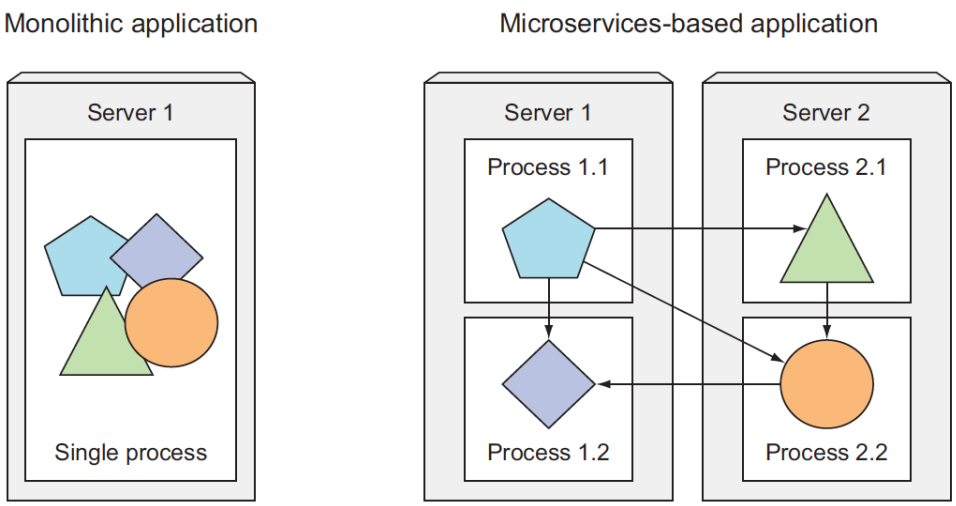
Introducción a Kubernetes **2**
Ayudar a los desarrolladores a enfocarse en las características principales de la aplicación 4
Ayudar a los equipos de operaciones a lograr una mejor utilización de los recursos 4
Arquitectura de un clúster de Kubernetes **4**
Ejecutando aplicaciones en K8s **5**
Beneficios de K8s **6**
Conceptos **7**
PODs 7
Replicación y otros controladores: implementando pods gestionados 9
Servicios: permitiendo que los clientes descubran y hablen con los pods 10
Ejecución de un clúster local de Kubernetes de un solo nodo con Minikube **10**

Kubernetes (K8S)

Introducción a Kubernetes

Kubernetes es una plataforma portátil, extensible y de código abierto para administrar cargas de trabajo y servicios en contenedores que facilita tanto la configuración declarativa como la automatización. Tiene un ecosistema grande y en rápido crecimiento. Kubernetes define un conjunto de bloques de construcción ("primitivas") que proporcionan colectivamente mecanismos que implementan, mantienen y escalan aplicaciones basadas en CPU, memoria o métricas personalizadas. Kubernetes está acoplado de forma laxa y es extensible para satisfacer diferentes cargas de trabajo. Permite a los desarrolladores implementar sus aplicaciones ellos mismos y con la frecuencia que deseen sin requerir ninguna ayuda del equipo de operaciones (ops). Kubernetes también ayuda al equipo de operaciones mediante la monitorización automática y la reprogramación de esas aplicaciones en caso de fallo del hardware.

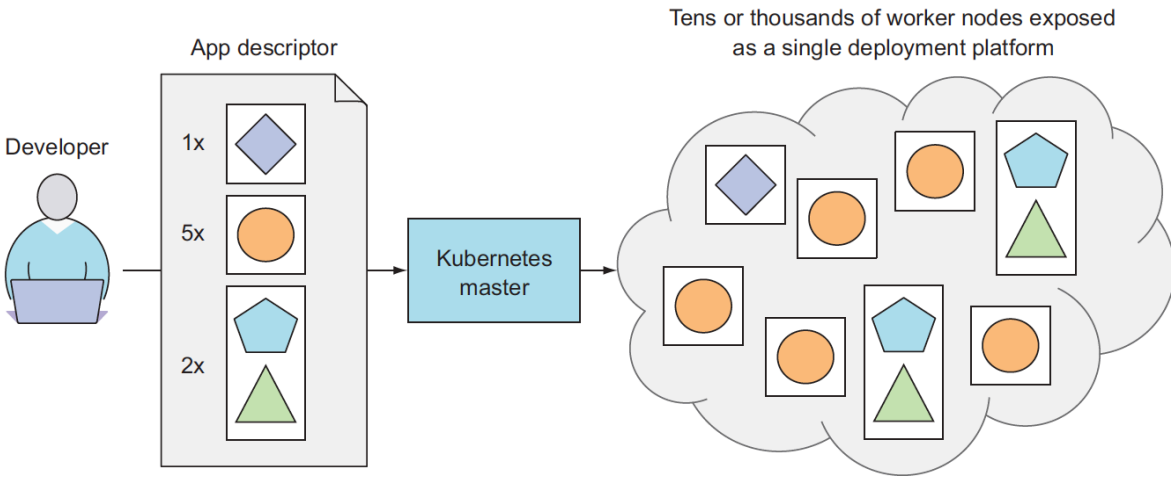
Aplicaciones Monolíticas y con Microservicios



A medida que aumenta el número de componentes de aplicaciones desplegables en un sistema, se vuelve más difícil administrarlos. Google fue una de las primeras empresas en darse cuenta de que necesitaba una forma mucho mejor de implementar y administrar sus componentes de software y su infraestructura a escala global. Debido a que opera cientos de miles de servidores, ha tenido que desarrollar soluciones para hacer que el desarrollo y la implementación de miles de componentes de software sean manejables y rentables.

Kubernetes permite ejecutar aplicaciones de software en miles de nodos de computadora como si todos esos nodos fueran un solo y enorme equipo. Abstrae la infraestructura subyacente y, al hacerlo, simplifica el desarrollo, la implementación y la administración tanto para los equipos de desarrollo como para los de operaciones.

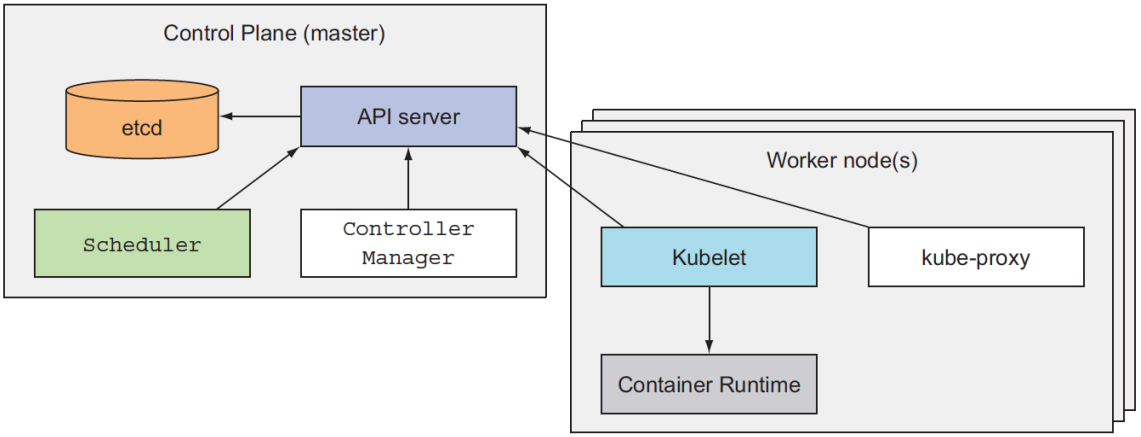
La implementación de aplicaciones a través de Kubernetes siempre es la misma, ya sea que su clúster contenga solo un par de nodos o miles de ellos. El tamaño del clúster no hace ninguna diferencia en absoluto. Los nodos adicionales del clúster simplemente representan una cantidad adicional de recursos disponibles para las aplicaciones implementadas.



Ayudar a los desarrolladores a enfocarse en las características principales de la aplicación

Ayudar a los equipos de operaciones a lograr una mejor utilización de los recursos

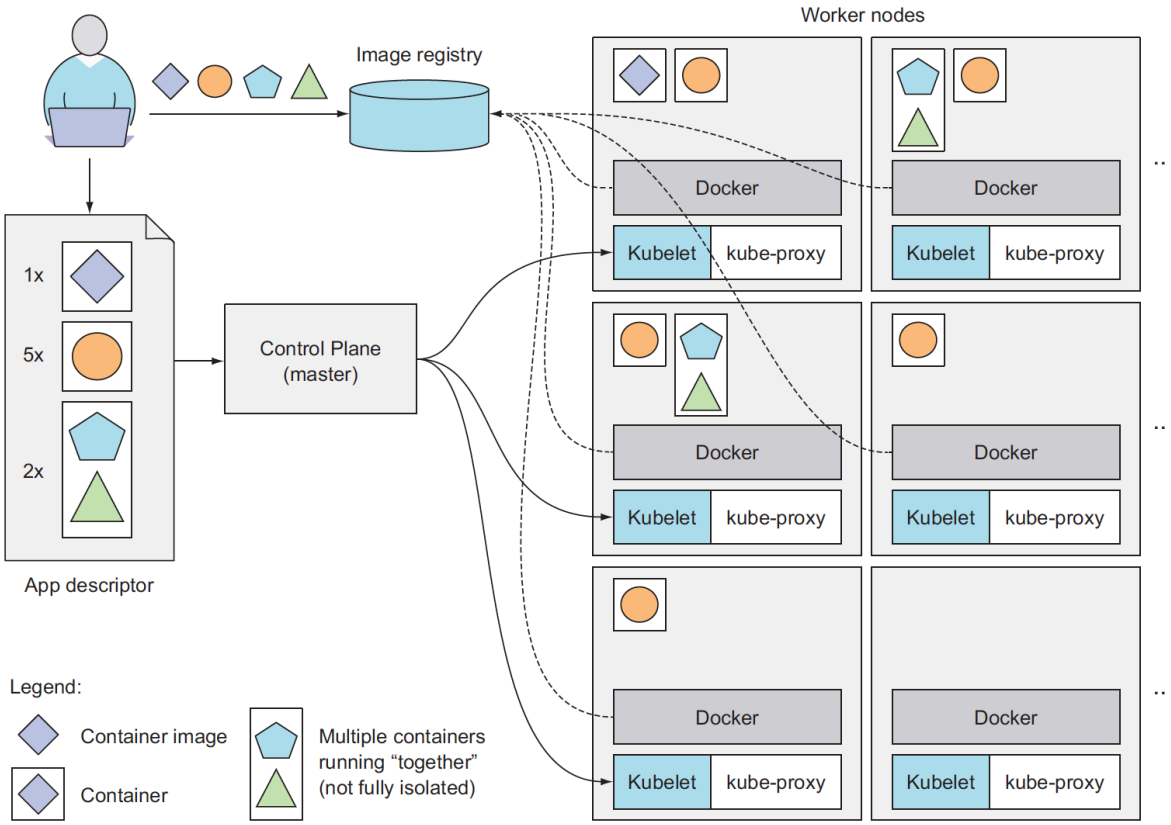
Arquitectura de un clúster de Kubernetes



- **Control Plane**: Es el conjunto de componentes que gestionan y controlan el clúster de Kubernetes. Incluye el API Server, el Control Manager y el Scheduler.

- **Nodes**: Son las máquinas virtuales o físicas en las que se ejecutan los contenedores. Cada nodo tiene un agente de Kubernetes llamado "kubelet" que se comunica con el Control Plane.
- **Pods**: Son la unidad básica de ejecución en Kubernetes. Cada pod consta de uno o más contenedores y comparten un conjunto común de recursos.
- **Servicios**: Son objetos de Kubernetes que definen un conjunto lógico de pods y proporcionan una IP y un nombre DNS único para acceder a ellos.

Ejecutando aplicaciones en K8s



Una vez que la aplicación se está ejecutando en Kubernetes, esté continuamente verifica que el estado actual de la aplicación coincida con la descripción proporcionada. Si especificas que siempre quieres que se ejecuten cinco instancias de un servidor web, Kubernetes mantendrá exactamente cinco instancias ejecutándose. Si una de estas instancias deja de funcionar correctamente, Kubernetes la reiniciará automáticamente. También puedes aumentar o disminuir el número de copias mientras la aplicación se está ejecutando, y Kubernetes creará o detendrá instancias adicionales según sea necesario.

Para que los clientes puedan encontrar fácilmente los contenedores que proporcionan un servicio específico, puedes indicarle a Kubernetes cuáles contenedores proporcionan el mismo servicio, y Kubernetes los expondrá a todos ellos en una única dirección IP estática. El kube-proxy se asegurará de que las conexiones al servicio se distribuyan entre todos los contenedores que proporcionan el servicio.

Beneficios de K8s

El equipo de operaciones ya no tiene que preocuparse por desplegar tus aplicaciones. Debido a que una aplicación contenerizada ya contiene todo lo que necesita para ejecutarse, los administradores del sistema no necesitan instalar nada para desplegar y ejecutar la aplicación. Kubernetes puede ejecutar la aplicación inmediatamente sin ayuda de los administradores del sistema.

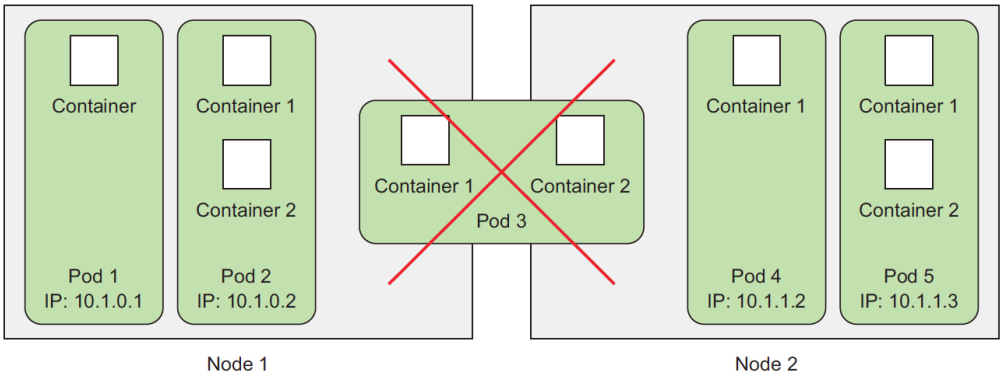
1. Simplificación en la implementación de aplicaciones: Al utilizar Kubernetes, los desarrolladores pueden implementar aplicaciones sin preocuparse por los servidores que conforman el cluster. Además, es posible especificar en qué tipo de nodo se desea que se ejecute cada aplicación.
2. Mejora en la utilización del hardware: Al utilizar contenedores, las aplicaciones pueden moverse libremente dentro del cluster, permitiendo que los componentes de la aplicación se ajusten y se empaqueten mejor en los nodos, optimizando la utilización del hardware disponible.
3. Verificación de estado y healthcheck: Kubernetes monitorea los componentes de las aplicaciones y los nodos en los que se ejecutan, reprogramando automáticamente los componentes a otros nodos en caso de fallos, permitiendo que el equipo de operaciones se centre en reparar los nodos en lugar de tener que reubicar las aplicaciones manualmente.
4. Escalado automático: Kubernetes puede monitorizar la carga de las aplicaciones y ajustar automáticamente el número de instancias en ejecución según sea necesario, permitiendo que el tamaño del cluster se ajuste automáticamente en función de las necesidades de las aplicaciones desplegadas.
5. Simplificación en el desarrollo de aplicaciones: Kubernetes facilita la implementación de ciertas características que de otra manera requerirían tiempo y esfuerzo, como la detección de servicios y compañeros en una aplicación agrupada, lo que permite que los desarrolladores se centren en otras tareas más importantes. Además, al usar

el mismo entorno en desarrollo y en producción, se pueden descubrir y solucionar errores más fácilmente.

Conceptos

PODs

Un pod en Kubernetes es un grupo de contenedores ubicados juntos y representa el bloque básico de construcción en Kubernetes. En lugar de implementar contenedores individualmente, siempre se implementan y se operan en un pod de contenedores. No estamos insinuando que un pod siempre incluye más de un contenedor, es común que los pods contengan solo un contenedor. Lo importante de los pods es que cuando un pod contiene múltiples contenedores, todos ellos se ejecutan en un solo nodo worker, nunca abarca varios nodos.



¿Pero por qué necesitamos pods? ¿Por qué no podemos usar contenedores directamente? ¿Por qué necesitaríamos ejecutar varios contenedores juntos? ¿No podríamos poner todos nuestros procesos en un solo contenedor? Responderemos esas preguntas ahora.

COMPRENDER POR QUÉ MÚLTIPLES CONTENEDORES SON MEJORES QUE UN CONTENEDOR QUE EJECUTA MÚLTIPLES PROCESOS

Comprendiendo los Pods

Dado que no se supone que agrupemos varios procesos en un solo contenedor, es obvio que necesitamos otro tipo de construcción de nivel superior que nos permita unir contenedores y administrarlos como una unidad única. Esta es la razón detrás de los pods.

Un pod de contenedores te permite ejecutar procesos estrechamente relacionados juntos y proporcionarles (casi) el mismo entorno como si todos estuvieran ejecutándose en un solo contenedor, al mismo tiempo que los mantiene algo aislados. De esta manera, obtienes lo mejor de ambos mundos. Puedes aprovechar todas las características que ofrecen los contenedores, al mismo tiempo que proporcionas a los procesos la ilusión de que están ejecutándose juntos.

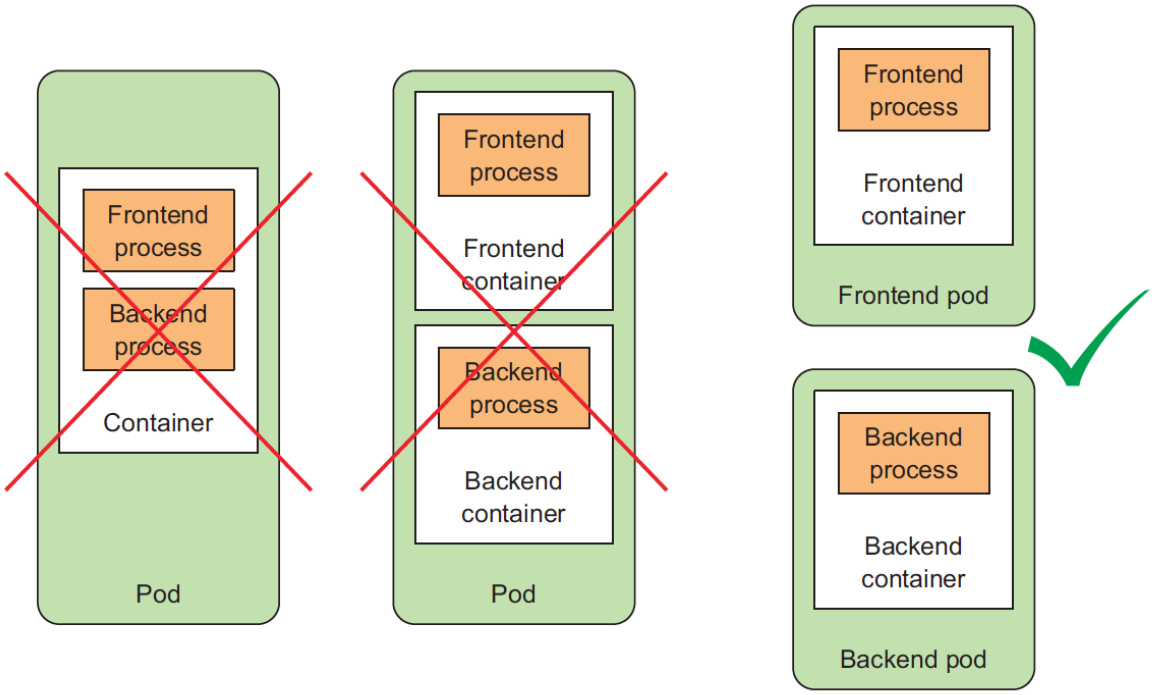


Figure 3.4 A container shouldn't run multiple processes. A pod shouldn't contain multiple containers if they don't need to run on the same machine.

DECIDIENDO CUÁNDO USAR MÚLTIPLES CONTENEDORES EN UN POD

Para resumir cómo se deben agrupar los contenedores en pods: al decidir si colocar dos contenedores en un solo pod o en dos pods separados, siempre debe hacerse las siguientes preguntas:

¿Necesitan ejecutarse juntos o pueden ejecutarse en diferentes hosts?

¿Representan un todo único o son componentes independientes?

¿Deben escalarse juntos o individualmente?

Básicamente, siempre debe gravitar hacia la ejecución de contenedores en pods separados, a menos que una razón específica requiera que sean parte del mismo pod.

Replicación y otros controladores: implementando pods gestionados

Una vez que tienes tus pods creados, necesitas asegurarte de que siempre haya suficientes instancias de ellos disponibles, para que tus aplicaciones estén siempre disponibles y sean resistentes a fallos. Para hacer esto, necesitas un controlador que pueda crear y gestionar automáticamente múltiples réplicas de tus pods.

En Kubernetes, un controlador que implementa esta funcionalidad se llama controlador de replicación. El controlador de replicación se encarga de crear y gestionar un conjunto de réplicas de un mismo pod. Si una réplica falla o se elimina por alguna razón, el controlador de replicación creará una nueva réplica para sustituirla.

El número de réplicas que el controlador de replicación mantiene se especifica en un objeto llamado ReplicationController, que es el que el controlador de replicación gestiona.

Pero hay otros controladores en Kubernetes además del controlador de replicación.

Estos incluyen:

- **ReplicaSet**: este controlador funciona igual que el controlador de replicación, pero se utiliza para hacer una distinción semántica cuando se trata de controlar aplicaciones que se ejecutan en un entorno de implementación de Kubernetes más moderno.
- **Deployment**: este controlador es el más utilizado para desplegar aplicaciones en Kubernetes. Permite implementar nuevas versiones de una aplicación de forma progresiva, sin tiempo de inactividad para el usuario. El controlador de replicación es una de las implementaciones subyacentes que utiliza el controlador de implantación para crear y gestionar réplicas de un pod.
- **DaemonSet**: este controlador se utiliza para asegurarse de que un pod se esté ejecutando en todos los nodos del clúster. Por ejemplo, si quieres implementar un agente de monitorización que recoja estadísticas de todos los nodos del clúster, puedes utilizar un DaemonSet para asegurarte de que haya una instancia de ese agente ejecutándose en cada uno de los nodos.

- **StatefulSet**: este controlador se utiliza para aplicaciones que requieren un orden de inicio y un orden de parada específicos, o que necesitan almacenamiento persistente.

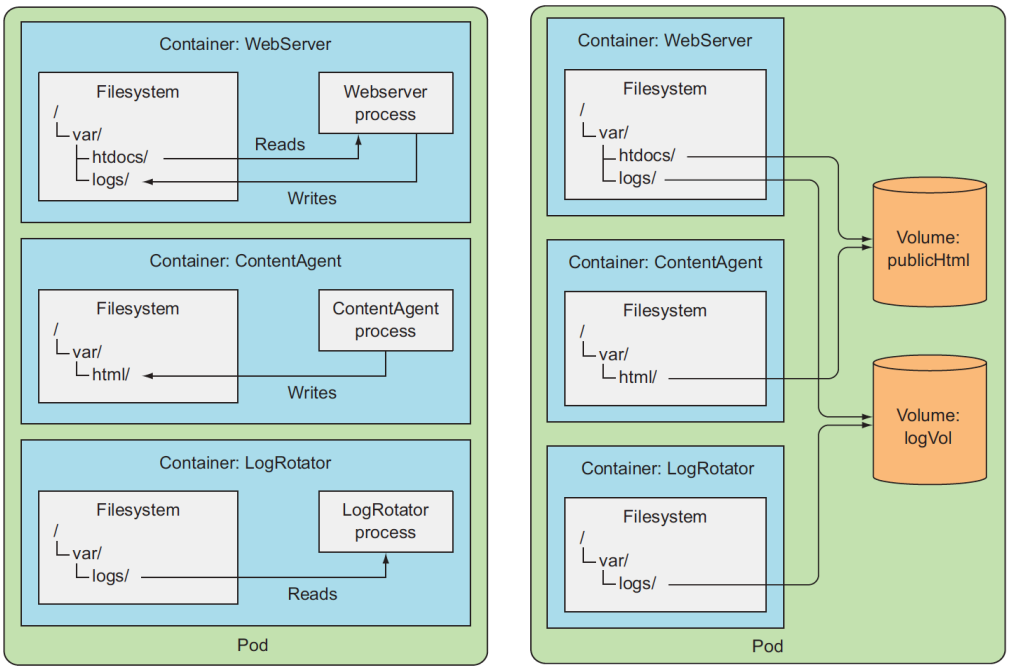
Servicios: permitiendo que los clientes descubran y hablen con los pods

Exponer Servicios para distintos Clientes:

- **NodePort**: el servicio de tipo NodePort en Kubernetes permite exponer un servicio a través de un puerto en cada nodo del clúster. Esto significa que cualquier tráfico que llegue a ese puerto en cualquiera de los nodos se redirigirá al servicio subyacente. Cuando se crea un servicio NodePort, se asigna automáticamente un número de puerto de nodo aleatorio en el rango 30000-32767. Si se desea, se puede especificar un número de puerto específico. Los clientes pueden acceder al servicio a través de la dirección IP de cualquier nodo del clúster y el puerto de nodo asignado al servicio. El servicio NodePort es una forma sencilla de exponer un servicio en un clúster de Kubernetes para su acceso externo, pero no es adecuado para entornos de producción en los que se necesita una dirección IP y un puerto fijo y estático. Para esos casos, se debe considerar el uso de un servicio de tipo LoadBalancer o una Ingress resource .
- **LoadBalancer**: una extensión del tipo NodePort: esto hace que el servicio sea accesible a través de un balanceador de carga dedicado, aprovisionado desde la infraestructura en la nube en la que se ejecuta Kubernetes. El balanceador de carga redirige el tráfico al puerto del nodo en todos los nodos. Los clientes se conectan al servicio a través de la dirección IP del balanceador de carga.
- **Ingress**, un mecanismo radicalmente diferente para exponer múltiples servicios a través de una sola dirección IP. Opera a nivel HTTP (capa de red 7) y, por lo tanto, puede ofrecer más características que los servicios de la capa 4.

Volumes

Los volúmenes en Kubernetes son un componente de un pod y se definen en la especificación del pod, al igual que los contenedores. No son un objeto independiente de Kubernetes y no se pueden crear ni eliminar por sí solos. Un volumen está disponible para todos los contenedores en el pod, pero debe ser montado en cada contenedor que necesite acceder a él. En cada contenedor, se puede montar el volumen en cualquier ubicación de su sistema de archivos.



Tipos de volúmenes

Hay una gran variedad de tipos de volúmenes disponibles en Kubernetes, algunos son genéricos y otros son específicos de tecnologías de almacenamiento subyacentes. A continuación, se presenta una lista de varios tipos de volúmenes disponibles:

- **emptyDir**: Un directorio vacío simple utilizado para almacenar datos transitorios.
- **hostPath**: Utilizado para montar directorios del sistema de archivos del nodo trabajador en el pod.
- **gitRepo**: Un volumen inicializado mediante la verificación del contenido de un repositorio Git.
- **nfs**: Un recurso NFS montado en el pod.

- **gcePersistentDisk**, awsElasticBlockStore, azureDisk: Utilizado para montar almacenamiento específico del proveedor de la nube.
- cinder, cephfs, iscsi, flocker, glusterfs, quobyte, rbd, flexVolume, vsphereVolume, photonPersistentDisk, scaleIO: Utilizado para montar otros tipos de almacenamiento en red.
- **ConfigMap** y **Secret**: Tipos especiales de volúmenes utilizados para exponer ciertos recursos y metadatos de Kubernetes al pod.
- **persistentVolumeClaim**: Una forma de usar almacenamiento persistente previamente o dinámicamente provisionado.

Cada tipo de volumen sirve para diferentes propósitos y se puede utilizar más de uno en un pod. Además, cada uno de los contenedores del pod puede tener el volumen montado o no.

EmptyDir

El tipo de volumen más simple es el emptyDir. Se trata de un directorio vacío que se crea cuando se inicia el pod y que se destruye cuando el pod finaliza. Puede ser utilizado por un solo contenedor cuando este necesita escribir datos temporalmente en el disco, como cuando realiza una operación de ordenación de un gran conjunto de datos. También puede ser utilizado para compartir archivos entre contenedores que se ejecutan en el mismo pod. Es útil para evitar la necesidad de replicar los mismos archivos en cada contenedor.

Persistencia de Almacenamiento

Cuando una aplicación en ejecución en un pod necesita persistir datos en disco y tener esos mismos datos disponibles incluso cuando el pod se vuelva a programar en otro nodo, no se puede utilizar ninguno de los tipos de volumen mencionados hasta ahora. Debido a que estos datos deben ser accesibles desde cualquier nodo del clúster, deben almacenarse en algún tipo de almacenamiento conectado en red (NAS).

```
apiVersion: v1
kind: Pod
metadata:
  name: mongodb
spec:
  volumes:
  - name: mongodb-data
    gcePersistentDisk:
      pdName: mongodb
      fsType: ext4
  containers:
  - image: mongo
    name: mongodb
    volumeMounts:
    - name: mongodb-data
      mountPath: /data/db
    ports:
    - containerPort: 27017
      protocol: TCP
```

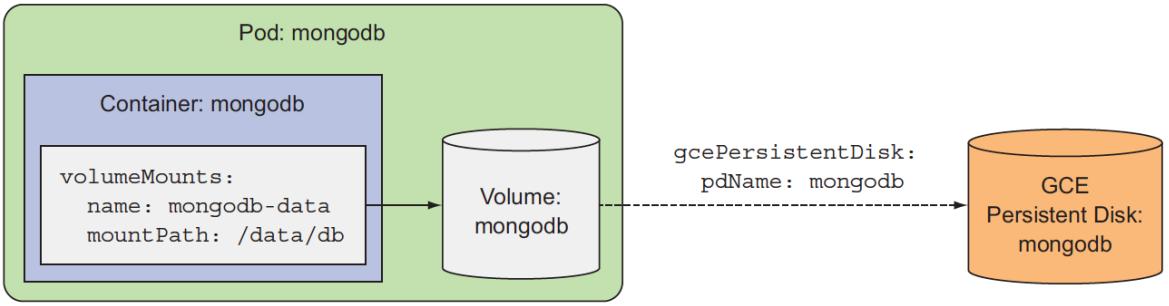
The name of the volume (also referenced when mounting the volume)

The type of the volume is a GCE Persistent Disk.

The name of the persistent disk must match the actual PD you created earlier.

The filesystem type is EXT4 (a type of Linux filesystem).

The path where MongoDB stores its data



PersistentVolumes y PersistentVolumeClaims

Para permitir que las aplicaciones soliciten almacenamiento en un clúster de Kubernetes sin tener que lidiar con detalles específicos de la infraestructura, se introdujeron dos nuevos recursos: PersistentVolumes y PersistentVolumeClaims. Los nombres pueden ser un poco confusos, porque como has visto en las secciones anteriores, incluso los volúmenes regulares de Kubernetes se pueden utilizar para almacenar datos persistentes.

El uso de un PersistentVolume dentro de un pod es un poco más complejo que el uso de un volumen regular de pod, por lo que ilustraremos cómo se relacionan los pods, PersistentVolumeClaims, PersistentVolumes y el almacenamiento subyacente real entre sí en la figura.

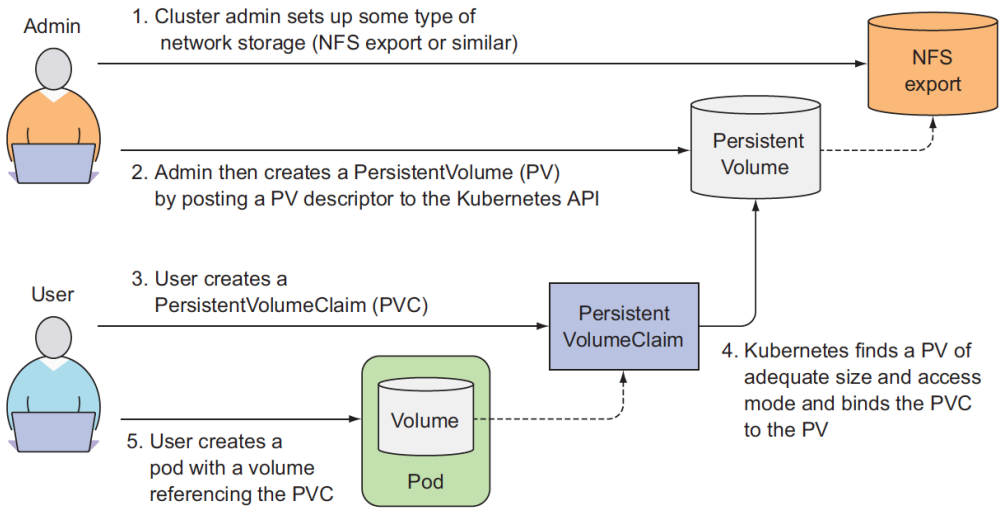


Figure 6.6 PersistentVolumes are provisioned by cluster admins and consumed by pods through PersistentVolumeClaims.

En lugar de que el desarrollador agregue un volumen específico de tecnología a su pod, es el administrador del clúster quien configura el almacenamiento subyacente y luego lo registra en Kubernetes mediante la creación de un recurso PersistentVolume a través del servidor API de Kubernetes. Al crear el PersistentVolume, el administrador especifica su tamaño y los modos de acceso que admite. Cuando un usuario del clúster necesita utilizar almacenamiento persistente en uno de sus pods, primero crea un manifiesto PersistentVolumeClaim, especificando el tamaño mínimo y el modo de acceso que requiere. Luego, el usuario envía el manifiesto PersistentVolumeClaim al servidor API de Kubernetes, y Kubernetes encuentra el PersistentVolume adecuado y lo enlaza con la solicitud. El PersistentVolumeClaim luego se puede utilizar como uno de los volúmenes dentro de un pod. Otros usuarios no pueden utilizar el mismo PersistentVolume hasta que se haya liberado eliminando el PersistentVolumeClaim vinculado.

Crear un PersistentVolume

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mongodb-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
  - ReadWriteOnce
  - ReadOnlyMany
  persistentVolumeReclaimPolicy: Retain
  gcePersistentDisk:
    pdName: mongodb
    fsType: ext4
```

Defining the PersistentVolume's size

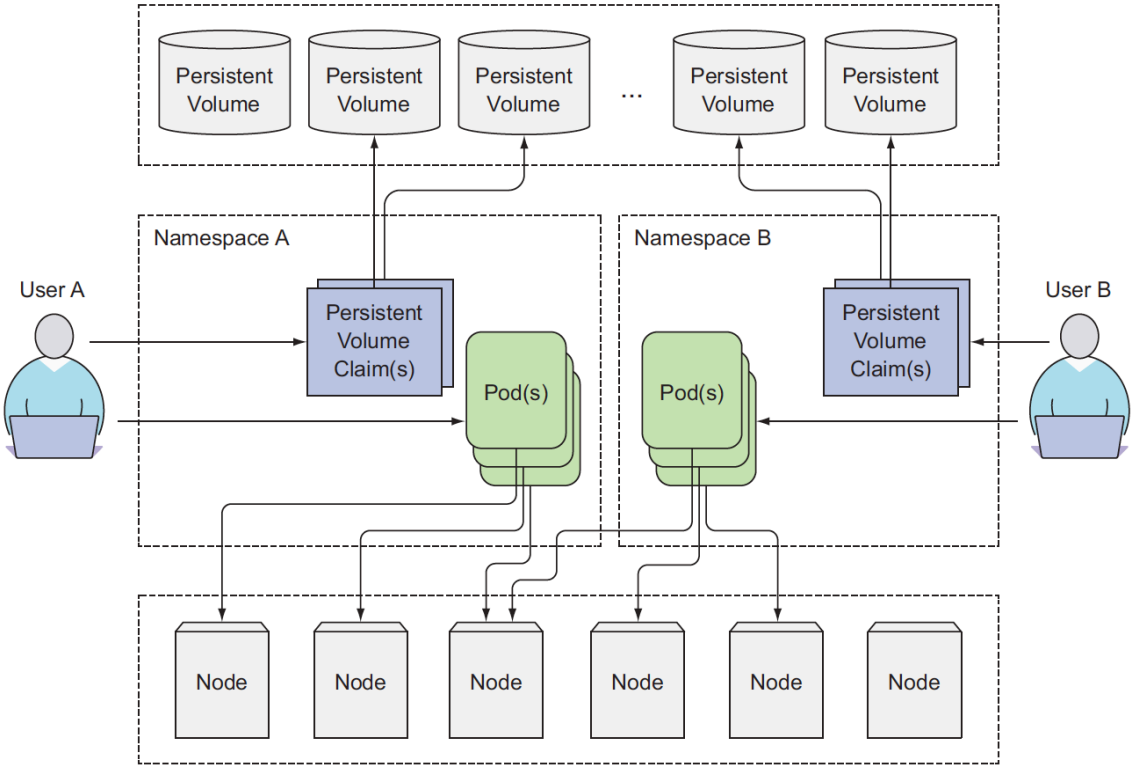
It can either be mounted by a single client for reading and writing or by multiple clients for reading only.

After the claim is released, the PersistentVolume should be retained (not erased or deleted).

The PersistentVolume is backed by the GCE Persistent Disk you created earlier.

```
spec:
  volumes:
  - name: mongodb-data
    gcePersistentDisk:
      pdName: mongodb
      fsType: ext4
```

NOTA: Los PersistentVolumes no pertenecen a ningún espacio de nombres (ver figura 6.7). Son recursos a nivel de clúster como los nodos.



Reclamando un PersistentVolume mediante la creación de un PersistentVolumeClaim.

Un PersistentVolume no puede ser usado directamente por un pod sin un PersistentVolumeClaim. Primero se debe crear el PersistentVolume y luego crear un PersistentVolumeClaim que lo reclame. El PersistentVolumeClaim es el objeto que solicita un espacio de almacenamiento específico en el PersistentVolume y una vez que se realiza la reclamación, se puede usar el PersistentVolumeClaim como un volumen dentro de un pod. La idea es que al separar la solicitud de almacenamiento de la definición del pod, el almacenamiento persistente se puede reutilizar por otros pods y no se pierde cuando el pod es destruido o recreado.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mongodb-pvc
```

The name of your claim—you'll need this later when using the claim as the pod's volume.


```
spec:
  resources:
    requests:
      storage: 1Gi
  accessModes:
  - ReadWriteOnce
  storageClassName: ""
```

Requesting 1 GiB of storage

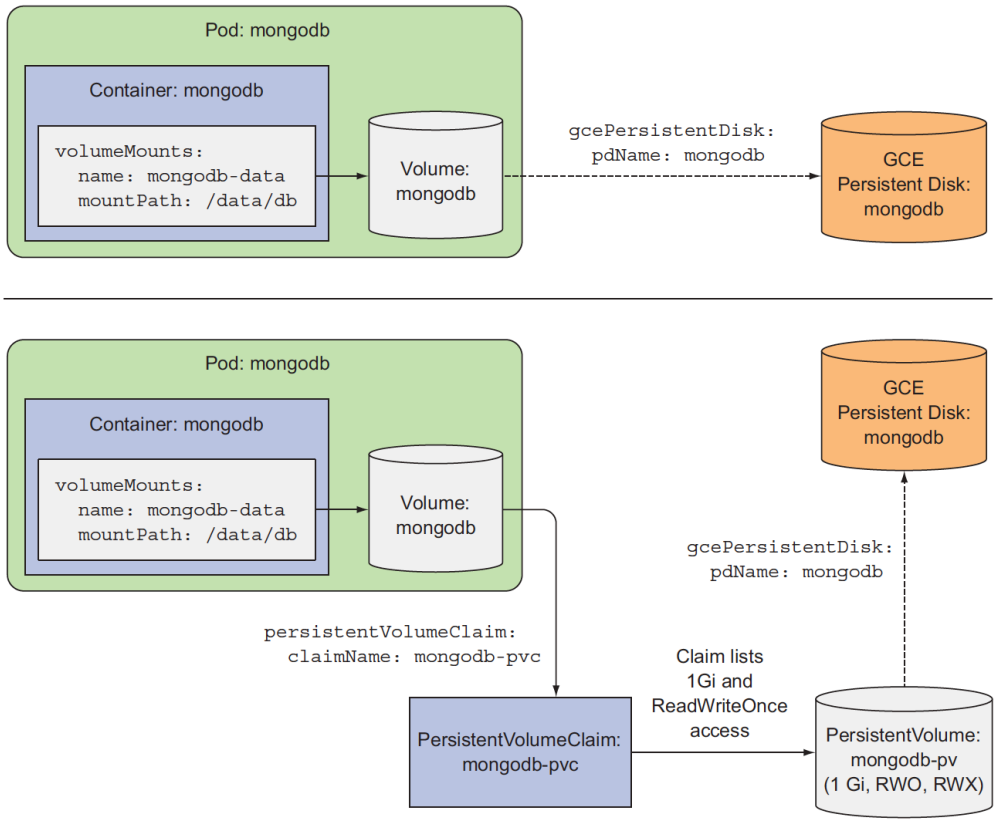
You want the storage to support a single client (performing both reads and writes).

You'll learn about this in the section about dynamic provisioning.

```
apiVersion: v1
kind: Pod
metadata:
  name: mongodb
spec:
  containers:
  - image: mongo
    name: mongodb
    volumeMounts:
    - name: mongodb-data
      mountPath: /data/db
    ports:
    - containerPort: 27017
      protocol: TCP
  volumes:
  - name: mongodb-data
    persistentVolumeClaim:
      claimName: mongodb-pvc
```

Referencing the PersistentVolumeClaim by name in the pod volume

Entendiendo los beneficios de usar PersistentVolumes y PersistentVolumeClaims.



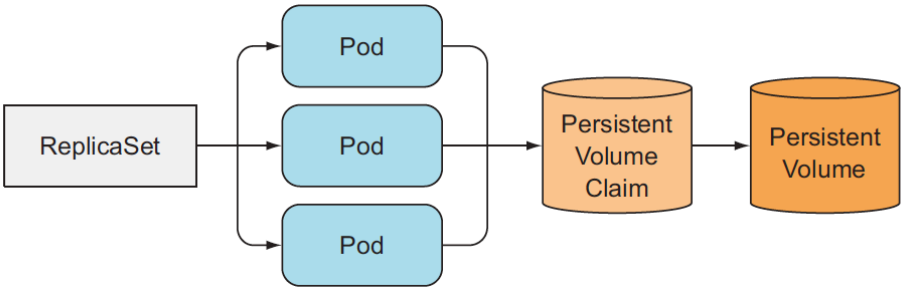
Usar este método indirecto de obtener almacenamiento de la infraestructura es mucho más simple para el desarrollador de la aplicación (o usuario del clúster). Aunque requiere los pasos adicionales de crear el PersistentVolume y el PersistentVolumeClaim, el desarrollador no necesita saber nada sobre la tecnología de almacenamiento utilizada en la infraestructura subyacente.

Además, los mismos manifiestos de pod y claim pueden ser utilizados en muchos clústeres de Kubernetes diferentes, ya que no se refieren a nada específico de la infraestructura. El claim indica: "Necesito x cantidad de almacenamiento y necesito poder leer y escribir en él por un solo cliente a la vez", y luego el pod hace referencia al claim por nombre en uno de sus volúmenes.

Statefulset

¿Se puede utilizar un ReplicaSet para replicar el pod de la base de datos?

Los ReplicaSets crean múltiples réplicas de un pod a partir de una sola plantilla. Estas réplicas no difieren entre sí, excepto por su nombre y dirección IP. Si la plantilla del pod incluye un volumen, que se refiere a una PersistentVolumeClaim específica, todas las réplicas del ReplicaSet utilizarán la misma PersistentVolumeClaim y, por lo tanto, la misma PersistentVolume ligada por la claim (mostrada en la figura).



Debido a que la referencia a la claim está en la plantilla del pod, que se utiliza para replicar múltiples réplicas de pod, no se puede hacer que cada réplica utilice su propia PersistentVolumeClaim separada. No se puede usar un ReplicaSet para ejecutar un almacén de datos distribuido, donde cada instancia necesite su propio almacenamiento separado, al menos no usando un solo ReplicaSet.

Alternativas:

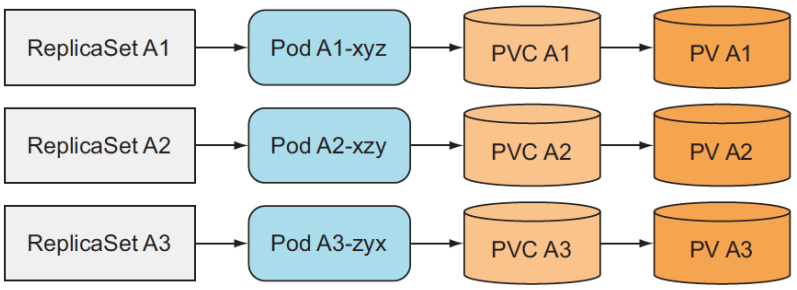


Figure 10.2 Using one ReplicaSet for each pod instance

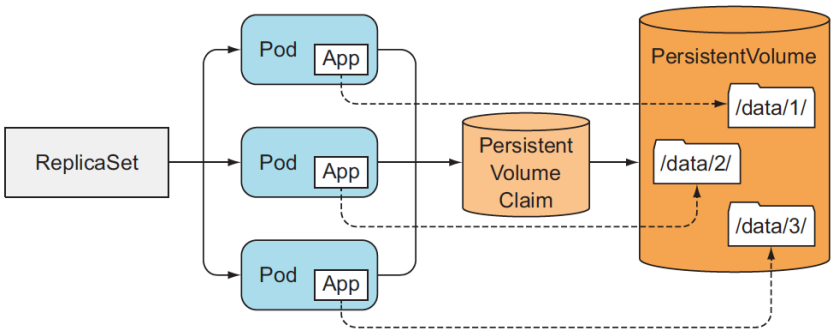


Figure 10.3 Working around the shared storage problem by having the app in each pod use a different file directory

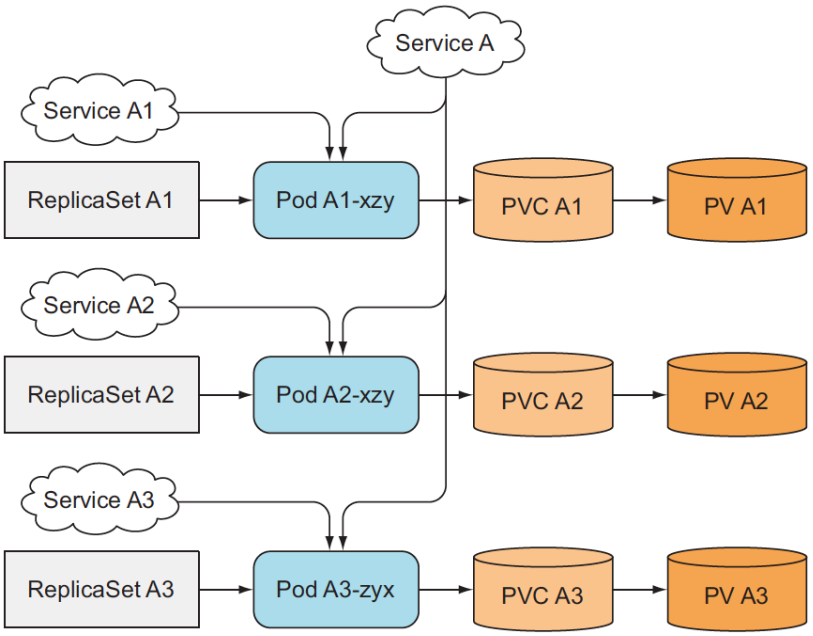


Figure 10.4 Using one Service and ReplicaSet per pod to provide a stable network address and an individual volume for each pod, respectively

Afortunadamente, Kubernetes nos salva de recurrir a soluciones tan complejas. La forma adecuada, limpia y sencilla de ejecutar estos tipos especiales de aplicaciones en Kubernetes es a través de un StatefulSet.

Explicación StatefulSets

En este tema se explica el concepto de StatefulSets, que son recursos específicamente diseñados para aplicaciones donde cada instancia de la aplicación debe tener un nombre y estado estables. Para entender su propósito, se comparan con ReplicaSets o ReplicationControllers, utilizando la analogía de mascota vs. ganado. Los

replicas de pods manejados por ReplicaSets o ReplicationControllers son como ganado, ya que son principalmente stateless y pueden ser reemplazados en cualquier momento. Sin embargo, para aplicaciones stateful, se necesita un enfoque diferente, ya que cuando una instancia muere, necesita ser resucitada en otro nodo con el mismo nombre, identidad y estado. Es ahí donde entra en juego StatefulSets, que garantiza que los pods sean reprogramados para retener su identidad y estado, y permite escalar el número de instancias arriba y abajo. A diferencia de los pods creados por ReplicaSets, los pods creados por StatefulSets no son réplicas exactas entre sí, cada uno puede tener su propio conjunto de volúmenes, lo que los diferencia de sus pares. Además, los pods tienen una identidad predecible y estable, en lugar de recibir una identidad completamente aleatoria.

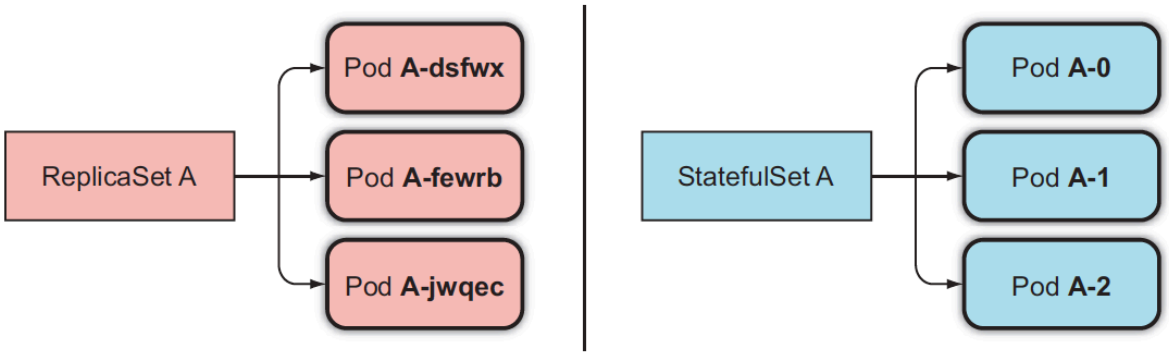


Figure 10.5 Pods created by a StatefulSet have predictable names (and hostnames), unlike those created by a ReplicaSet

Servicio de gobernanza

A diferencia de los pods regulares, los pods con estado a veces necesitan ser direccionables por su nombre de host, ya que cada pod puede contener un estado diferente. El servicio de gobernanza es utilizado para proporcionar una identidad de red real a cada pod a través de su propia entrada DNS, lo que permite a los clientes en el clúster abordar el pod por su nombre de host.

Para este propósito, un StatefulSet requiere que se cree un Servicio ***headless*** gobernante correspondiente que se utiliza para proporcionar la identidad de red real a cada pod. A través de este Servicio, cada pod obtiene su propia entrada DNS, por lo que sus pares y posiblemente otros clientes en el clúster pueden dirigirse al pod por su nombre de host. Por

ejemplo, si el Servicio gobernante pertenece al espacio de nombres predeterminado y se llama "foo", y uno de los pods se llama "A-0", puede acceder al pod a través de su nombre de dominio completo, que es "a-0.foo.default.svc.cluster.local".

Además, también puede utilizar DNS para buscar los nombres de todos los pods del StatefulSet buscando registros SRV para el dominio "foo.default.svc.cluster.local".

Reemplazando Mascotas perdidas

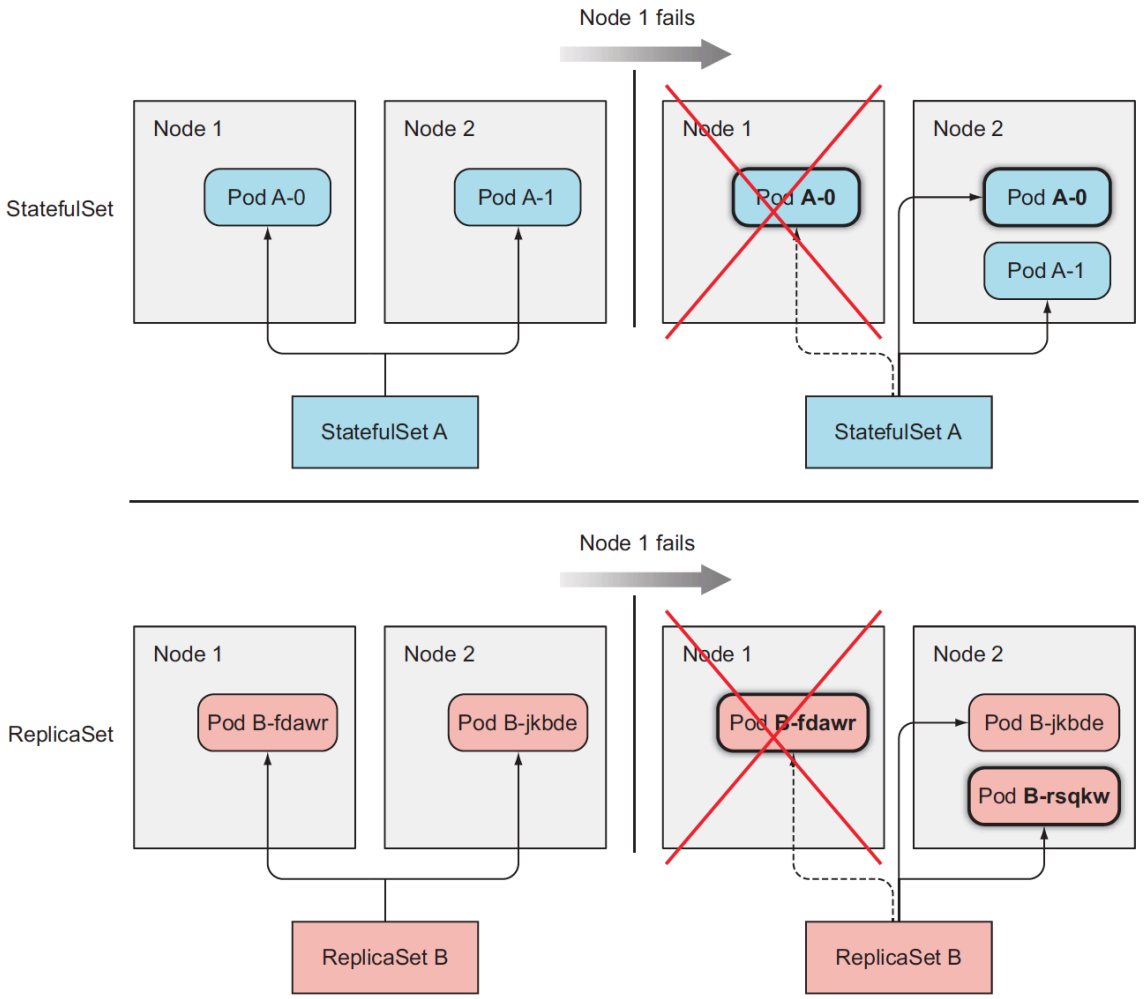


Figure 10.6 A StatefulSet replaces a lost pod with a new one with the same identity, whereas a ReplicaSet replaces it with a completely new unrelated pod.

Escalando Statefulset

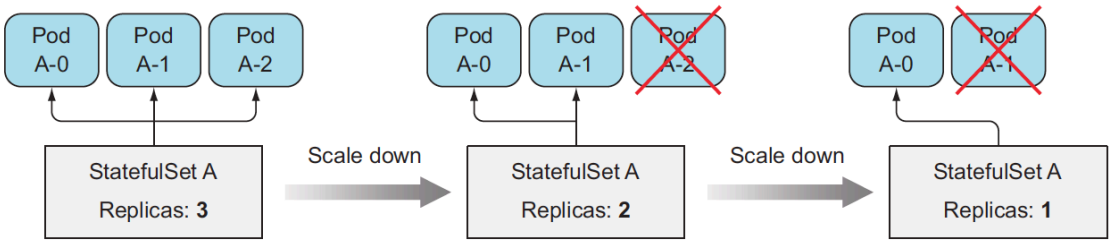


Figure 10.7 Scaling down a StatefulSet always removes the pod with the highest ordinal index first.

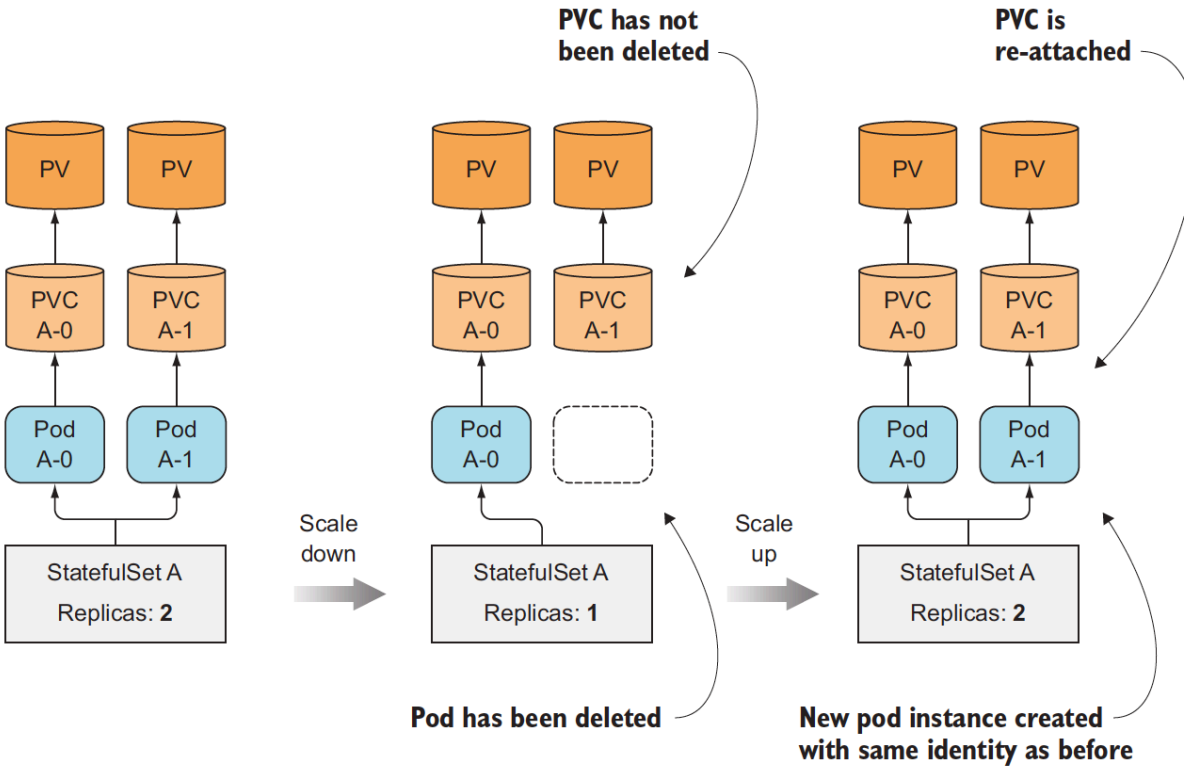


Figure 10.9 StatefulSets don't delete PersistentVolumeClaims when scaling down; then they reattach them when scaling back up.

Ejecución de un clúster local de Kubernetes de un solo nodo con Minikube

https://gitlab.com/public-unrn/k8s